

Finite-Catalog Resource Allocation: Exact Path Representations and Menu Complexity

Anonymous manuscript for Operations Research — R59

Abstract. A finite command catalog couples continuous resource allocation to the cost and cardinality of its implementation. We study a renewal design problem with an accepted aggregate promise, individual expected caps, realization ceilings, and service chosen before a terminal draw. A constructive reduction yields exact path capacities and concave edge payoffs. We characterize the capacity-conservation property on acyclic graphs and identify when all starting points share one deficit state. Within the original model, exact optimization is parameterized-hard jointly in command allowance and cap count, even with a common linear reward and free service. This separates small-type enumeration from fixed-parameter tractability. A centered deficit algorithm nevertheless provides an additive value interval while preserving the original promise and selected commands. Cap-preserving prototype optimization gives a complementary menu-loss frontier. Computational evidence distinguishes rational certificates from numerical mixed-integer convex bounds, includes verification cost, and retains unsuccessful runs. The results identify a structural and encoding-sensitive frontier for finite-catalog allocation rather than assuming that a short command book makes exact design easy.

Keywords: resource allocation; dynamic programming; parameterized complexity.

Subject classifications: Programming: resource allocation; Dynamic programming: finite-catalog design.

Area of review: Optimization.

1 Introduction

A finite command catalog creates a distinction between the resources allocated to a decision and the number of commands used to implement it. A service provider may deliver different expected obligations to different customer histories while installing only a small common set of terminal commands. Installing a command incurs a fixed charge. Each history also has an expected cap and a ceiling on every realized obligation. Optimizing the expected targets without optimizing their shared implementation can therefore miss both a discrete cost and a feasibility restriction.

We study this joint allocation and implementation problem, preserving the accepted aggregate promise and deterministic service chosen before the terminal draw. Our contribution is a structural theory rather than an empirical claim about a particular provider. The service-credit interpretation makes the timing and resource accounting concrete. It does not supply customer-choice, incentive-compatibility, or field-calibration assumptions that are absent from the model.

The analysis has three connected parts. First, we eliminate the history-level lotteries constructively and obtain an exact resource allocation problem on a selected path. Each edge has a capacity that is the difference of two values of a common potential. We characterize this conservation property on arbitrary acyclic graphs, give a linear-time recognition algorithm with a path witness when it fails, and identify exactly when complementing resources permits all starting vertices to share one terminal deficit. Thus the useful abstraction is not a renewal-specific identity alone: it is a recognizable class of endogenous path and resource-allocation problems.

Second, we establish a limited-menu complexity frontier inside the original model. Exact optimization is hard in the parameterized sense even when both the command allowance and the number of distinct expected caps are parameters. The reduction retains a common linear reward, free service, uniform probabilities, equal expected caps and realization ceilings, and nonnegative opening charges. Its dedicated baseline histories force the baseline commands to consume the actual budget. This closes the gap between a nonbinding-budget subset-sum construction and the previously established small-type enumeration bound. The lower bound does not contradict polynomial algorithms for a fixed parameter value or numerical-lattice algorithms: it distinguishes their dependence on parameters and encoding.

Third, resource conservation supports an original-feasible additive approximation. A single deficit coordinate merges all feasible starting points. Rounding and repairing on the same selected path preserves the exact promise and the command count. Centered concave edge kernels permit fast convolution even when intermediate states contain unreachable gaps. The abstract theorem covers conservative acyclic paths; the renewal model supplies an exact implementation of its edge allocations. Optimal prototype selection in a declared reward family makes the type approximation operational. Joint reward types, exact lattice regimes and price-based certificates provide complementary refinements in the electronic companion.

The computational analysis is organized around what these results establish: the cost of representing, solving, and checking a decision, not universal superiority over mixed-integer optimization. We preserve all earlier failures and timings and distinguish them from new executions. A direct mixed-integer convex formulation has no service-envelope approximation. Its numerical solver bounds are reported separately from independently checked rational intervals. Screening is treated as an optional preprocessing decision whose

cost must be included; its unfavorable earlier matched comparison is retained. The article consequently makes a theory claim supported by algorithmic evidence, not an unverified claim of operational adoption.

1.1 Relationship to prior work

Constrained-path methods provide the algorithmic context. Lagrangian bounds and constrained shortest-path search are established tools (Handler and Zang, 1980; Beasley and Christofides, 1989; Fisher, 1981). Approximation schemes for restricted shortest paths also precede this work (Hassin, 1992; Lorenz and Raz, 2001). Those results do not themselves eliminate our individual expected and realized constraints. Once an exact path representation has been obtained, however, resource-state dynamic programming and Lagrangian branching are antecedent machinery, not new inventions. Our contribution at that interface is the implementable edge model, a testable source-conservation criterion, and a repair that keeps the selected implementation and original equality.

A fixed path leads to separable nonlinear resource allocation; its continuous allocation step belongs to the literature surveyed by Patriksson (2008). Concave convolution uses matrix-searching structure (Aggarwal et al., 1987); our finite-support argument explains why unreachable holes do not invalidate that use. The new parameterized reduction, rather than the existence of a resource dynamic program, establishes the discrete frontier. The relevant distinction between a parameter-dependent exponent and fixed-parameter tractability follows Downey and Fellows (1995).

Small support and a small shared catalog are different objects. Extreme-point results for moment-constrained probability measures bound the support of a distribution under specified moment constraints (Winkler, 1988). They do not bound the union of history-dependent supports when the union incurs opening charges. Auction menu complexity instead counts outcome–payment alternatives subject to incentive constraints (Hart and Nisan, 2013; Babaioff et al., 2016). Our terminal commands carry neither a free history label nor a payment menu; histories are observed by the allocator, not selected strategically by a buyer. These distinctions prevent a support bound or an auction approximation theorem from being imported as a proof of our cap-count result.

There is also a distinction from assortment and data-driven optimization. In choice-based assortment optimization, offering an item changes demand through the choice model, and empirical demand learning may be part of the decision problem (Rusmevichientong et al., 2010). Here the history probabilities and accepted promise are model inputs and command probabilities are controlled decisions. Distributionally robust optimization addresses uncertainty in distributions and provides statistical or worst-distribution guarantees under explicit ambiguity models (Mohajerin Esfahani and Kuhn, 2018); our reward-oscillation certificate holds feasibility inputs fixed and is not such a guarantee. Finally, prototype selection is a clustering problem. We identify its precise oscillation distance and cap restrictions, use the standard facility-location formulation for general medoids, and solve a one-dimensional common-curvature case by weighted-median dynamic programming. We do not claim a new general metric clustering algorithm (Arya et al., 2004; Wu, 1991).

Table 1 states the nearest theorem-level comparisons. Stripping away every certificate, build manifest, and repository artifact leaves the constructive representation, the original-model parameterized hardness theorem, the conservative-path recognition and approximation results, and the loss-controlled implementation frontier.

Reproducibility supports these claims but is not a substitute for them.

2 Finite-Catalog Renewal Design

There are k histories. History j has probability $\pi_j > 0$, expected-obligation cap $b_j \in [0, 1]$, and realization ceiling $\tau_j \geq b_j$, with $\sum_j \pi_j = 1$. Write $\bar{b} = \sum_j \pi_j b_j$. Intermediate service has cost $h_j : [0, b_j] \rightarrow \mathbb{R}_+$, continuous, convex and nondecreasing, with $h_j(0) = 0$. The initial common terminal reward $f : [0, 1] \rightarrow \mathbb{R}$ is strictly increasing and concave. The catalog is $\mathcal{A} = \{a_1 < \dots < a_N\} \subset [0, 1]$. Opening command a_i costs $\rho_i \geq 0$.

A nonempty selected book $c = (c_1 < \dots < c_s)$ has $s \leq m$. After observing j , the provider selects deterministic pre-draw service $y_j \geq 0$ and a lottery C_j supported on c . The problem is

$$V^*(B) = \max_{c, y, C} \left\{ \sum_j \pi_j [\mathbb{E}f(C_j) - h_j(y_j)] - \sum_{z \in c} \rho(z) \right\}, \quad (1)$$

$$t_j = y_j + \mathbb{E}C_j \leq b_j, \quad y_j + C_j \leq \tau_j \quad \text{almost surely}, \quad \sum_j \pi_j t_j = B. \quad (2)$$

Here $B \in [0, \bar{b}]$ is accepted, rather than optimized away. The common-risk specialization is $\tau_j = b_j + \delta$. Expected and realized obligations are different restrictions. The terminal symbol does not carry a free history index, inherited service tier, or shared random seed. The cardinality m measures the command alphabet, not total implementation storage: targets, encoder logic, probability precision, program constants, and certificates are additional resources. The mathematical results below concern the stated finite catalog; continuous-design results have a separate approximation error.

For a feasible book, let $d_j = \max(c \cap [0, \tau_j])$, and let $F_{j,c}$ interpolate f linearly on this eligible prefix. A singleton prefix has the constant value $f(d_j)$ at its only point.

Proposition 2.1 (Canonical response). *A book is feasible if and only if $c_1 \leq \min_j b_j$ and $c_1 \leq B \leq \bar{b}$. At target $c_1 \leq t \leq b_j$, its optimal response is*

$$v_{j,c}(t) = F_{j,c}(\min\{t, d_j\}) - h_j((t - d_j)_+). \quad (3)$$

It is attained by $y_j = (t - d_j)_+$ and an adjacent eligible lottery with mean $\min(t, d_j)$. The response is concave in t .

Proof. Every feasible command satisfies $C_j \leq \tau_j$ since $y_j \geq 0$. At mean $\mu = \mathbb{E}C_j$, concavity on the ordered support bounds the reward by $F_{j,c}(\mu)$. The payoff is at most $F_{j,c}(\mu) - h_j(t - \mu)$ for $c_1 \leq \mu \leq \min(t, d_j)$. Both terms are nondecreasing in μ , so the upper endpoint maximizes this bound. When $t \leq d_j$, adjacent interpolation has zero intermediate service and support below τ_j . When $t > d_j$, use command d_j surely and service $t - d_j$; the realized sum is $t \leq b_j \leq \tau_j$. Thus the bound is attained without weakening either restriction. Interpolation slopes decrease and are nonnegative; the tail slopes of $-h_j$ are nonpositive and nonincreasing. This proves concavity. Feasible targets fill the entire weighted interval $[c_1, \bar{b}]$, proving necessity and sufficiency. $\square$

The algorithmic specialization is

$$f(x) = rx - \frac{1}{2}qx^2, \quad r \geq q \geq 0, \quad r > 0, \quad h_j(y) = \frac{1}{2}\gamma_j y^2, \quad \gamma_j \geq 0. \quad (4)$$

All probabilities, caps, ceilings, catalog points, shape parameters, opening charges, and requested accuracies are rationally encoded in exact-arithmetic claims. Optimization identities allow real nonnegative charges; bit-complexity assertions do not assign a finite encoding to arbitrary real numbers. A common Lipschitz constant for the canonical responses is

$$L = \max\{r, \max_j \gamma_j\} > 0. \quad (5)$$

Continuity of the cost on its compact domain makes the stated extrema attained; every rational quadratic instance already satisfies this regularity. A fixed-book allocation can be solved by filling positive chord slopes in decreasing order, then zero-cost service capacity, then quadratic service tails. The service breakpoints are $\gamma_j(b_j - d_j)_+$. This takes $O(ks + k \log(k+1))$ rational operations with sorted marginal events. Alternatively, a direct supporting-price sweep verifies

$$\sum_j \pi_j v_{j,c}(t_j) = \lambda B + \sum_j \pi_j \max_{c_1 \leq t \leq b_j} \{v_{j,c}(t) - \lambda t\}. \quad (6)$$

Eligible knots, caps, and clipped stationary tail points suffice to check each scalar maximum. This certificate proves optimality for that book, not over all books.

2.1 Infeasible inputs and certificate scope

For well-formed primitives and a nonempty catalog, feasibility is decided before an incumbent is initialized. If $m < 1$, $B < 0$, $B > \bar{b}$, or $a_1 > \min\{B, \min_j b_j\}$, no feasible book exists. Necessity follows from nonemptiness, the aggregate caps and the minimum possible command; sufficiency follows by using the singleton a_1 and filling deterministic service to the accepted promise. An empty catalog is likewise infeasible. A malformed probability vector, unsorted catalog or invalid primitive is instead an input error and must not be reported as an optimization certificate.

An infeasibility record identifies the failed inequality and carries the external input digest. It has no incumbent, lower policy or finite objective interval. An independent checker recomputes the probability-weighted cap, minimum cap and minimum catalog point from the supplied original input. Validating a scalar inequality is sufficient; search is unnecessary. The current solver modules retain their legacy exception on infeasible requests, whereas `feasibility59.py` provides this explicit preflight record and checker. The new panel is generated feasible and does not count preflight rejection as an optimization success.

Units and loss. Obligations and caps use one normalized resource unit. Rewards, service costs and installation charges share a net-value unit. An additive tolerance is stated in that value unit; rescaling every payoff and the tolerance by the same positive factor preserves its decision meaning. The present theory makes no empirical currency calibration.

3 Constructive Selected-Boundary Decomposition

This section assumes the common reward f in Section 2. For a selected book $c = (u_1 < \dots < u_s)$, write $a = u_1$ and $d_j = \max\{u_i : u_i \leq \tau_j\}$.

Lemma 3.1 (Global terminal-first allocation). *An optimal fixed-book canonical allocation cannot have both $t_i > d_i$ and $t_j < \min(b_j, d_j)$.*

Proof. Choose $0 < \delta \leq \min\{t_i - d_i, \pi_j[\min(b_j, d_j) - t_j]/\pi_i\}$. Reduce t_i by δ and increase t_j by $\pi_i\delta/\pi_j$. The weighted change is zero. The donor's terminal reward is unchanged and its service cost weakly decreases; the recipient's strictly increasing terminal interpolant improves. Both responses remain feasible by Proposition 2.1, contradicting optimality. $\square$

For selected neighbors $u < v$, define

$$\delta_j(u, v) = \mathbf{1}_{\{\tau_j \geq v\}} [\min(b_j, v) - u]_+, \quad T_{uv} = \sum_j \pi_j \delta_j(u, v), \quad (7)$$

$$J_{uv} = \{j : u \leq \tau_j < v\}, \quad Q_{uv} = \sum_{j \in J_{uv}} \pi_j (b_j - u)_+. \quad (8)$$

The capped service value on an exit set is

$$H_{uv}(y) = \min \left\{ \sum_{j \in J_{uv}} \pi_j h_j(z_j) : 0 \leq z_j \leq (b_j - u)_+, \sum_{j \in J_{uv}} \pi_j z_j = y \right\}, \quad 0 \leq y \leq Q_{uv}. \quad (9)$$

Empty sets have zero capacity and zero value at zero. Let $\sigma_{uv} = [f(v) - f(u)]/(v - u)$ and

$$\Phi_{uv}(x) = \sigma_{uv} \min(x, T_{uv}) - H_{uv}((x - T_{uv})_+), \quad 0 \leq x \leq T_{uv} + Q_{uv}. \quad (10)$$

For the terminal edge $(u, \dagger)$, use $T_{u\dagger} = 0$, $J_{u\dagger} = \{j : \tau_j \geq u\}$, and $\Phi_{u\dagger} = -H_{u\dagger}$. The functions Φ are concave. Under the quadratic primitives, they are L -Lipschitz with L as in (5).

Lemma 3.2 (Nested terminal layers). *Start each history at target a . Fill the selected terminal layers in increasing edge order. Once every preceding layer is full, each history with $\delta_j(u, v) > 0$ is at target u . Every weighted amount $x \in [0, T_{uv}]$ in the next layer has a feasible individual allocation. After all terminal layers are full, history j is at $\min(b_j, d_j)$; its remaining service capacity belongs to exactly one exit set.*

Proof. A history with positive increment on (u, v) has $b_j > u$ and $\tau_j \geq v$. It is eligible for all earlier selected commands, and the increments of those earlier layers telescope to $u - a$. This proves the first assertion, including a history whose cap will truncate the current layer.

For any desired x , initialize residual $r = x$ and visit histories in any fixed order. Give history j increment $e_j = \min\{\delta_j(u, v), r/\pi_j\}$ and reduce r by $\pi_j e_j$. Since $x \leq \sum_j \pi_j \delta_j(u, v)$, the final residual is zero. Each affected target lies in $[u, \min(b_j, v)]$ and is implemented by the adjacent lottery on u, v with upper probability $e_j/(v - u)$. It has no intermediate service and both realizations are below τ_j . Histories with zero increments keep their previously feasible responses.

If a layer is partial, stop: its predecessors are full and its successors unused. If every layer is full, the increments for history j telescope to $\min(b_j, d_j) - a$. It exits exactly after d_j , at an ordinary crossing edge or at the terminal edge. Thus its remaining capacity is $(b_j - d_j)_+$ and appears in exactly one H . Any feasible service vector on the exit sets implements target $\min(b_j, d_j) + z_j$. A positive z_j requires $b_j > d_j$, and its implementation is the sure command d_j plus service z_j ; their sum is at most $b_j \leq \tau_j$. Zero-capacity histories require no service. The exit sets are disjoint, so these implementations combine without conflicting allocations. $\square$

Lemma 3.3 (Implementable rearrangement). *For a fixed selected path, any feasible arc-resource vector can be rearranged, with unchanged total resource and no smaller payoff, into full earlier terminal layers, at most one partial terminal layer, and service only after all terminal layers are full. That rearranged payoff is attained by original history-level policies.*

Proof. Split the resource on each arc into its terminal and service parts in (10). If any terminal capacity remains unused while some service is positive, transfer the smaller of the available terminal capacity and a positive service amount to that terminal layer. The terminal gain is positive, while reducing service weakly lowers cost. Repeat until there is no such pair. This operation never changes charges or the sum of resources.

Next, if an earlier terminal layer is unfinished and a later one is positive, transfer the smaller of the unfinished capacity and the later mass. Selected chord slopes are positive and nonincreasing, so payoff cannot decrease. Filling layers successively yields the asserted form after finitely many exhausted donors or filled recipients. Each arc still respects its capacity: terminal mass is at most T_e , and service, when present, is at most Q_e after that arc's terminal part is full. Allocate any remaining service by minimizers of the corresponding H_e ; these minimizers exist by compactness and continuity. Lemma 3.2 implements the resulting terminal pattern and the disjoint service allocations. Its gross payoff is precisely $f(a) + \sum_e \Phi_e(x_e)$ for the rearranged vector. An arbitrary original vector need not be implemented at its unrearranged payoff; the weak improvement is the comparison needed for exact optimal values. $\square$

Theorem 3.4 (Exact selected-boundary resource path). *Let $P(c)$ contain the successive selected edges and the terminal edge of a feasible book. Then*

$$\sum_{e \in P(c)} (T_e + Q_e) = \bar{b} - a, \quad (11)$$

$$V_c(B) = f(a) - \sum_{z \in c} \rho(z) + \max_{\substack{0 \leq x_e \leq T_e + Q_e \\ \sum_e x_e = B - a}} \sum_{e \in P(c)} \Phi_e(x_e). \quad (12)$$

Maximizing over paths with at most m selected commands solves the original joint problem exactly. Reconstruction fills terminal layers in order, uses the greedy partial-layer allocation of Lemma 3.2, and then solves the disjoint capped service allocations. The resulting policy obeys the original promise, expected caps, realization ceilings, and command budget.

Proof. For history j , the terminal increments sum to $\min(b_j, d_j) - a$ and its unique service capacity is $(b_j - d_j)_+$. Their sum is $b_j - a$, proving (11) after weighting. In an original optimum, Lemma 3.1 places

service after terminal capacity. Among terminal layers, exchanging equal weighted amounts orders the common decreasing chord slopes; Lemma 3.2 justifies the individual feasibility of that ordering. If all terminal capacity is full, the exit-set cost minimizations describe all remaining feasible service allocations. Hence an original optimum supplies an arc allocation of the same value. Conversely, Lemma 3.3 implements a weakly improved version of every feasible arc allocation. The two comparisons yield equality of the attained maxima. Each selected charge is subtracted once, and the terminal edge does not open an additional command. $\square$

Corollary 3.5 (Endogenous eligibility). *A book of s commands has at most s nonempty selected-prefix classes. Inserting unselected candidates changes neither its arc functions nor its fixed-book optimum.*

Proof. The nonempty exit sets partition histories according to their last eligible selected command. Their definitions and terminal increments depend on the selected endpoints, not intervening unused candidates. $\square$

3.1 An eligibility-independent capacity potential

The path-capacity identity has a stronger local form. It explains why eligibility affects returns, but does not create a hidden resource state.

Proposition 3.6 (Capacity potential). *Define $p(u) = \sum_j \pi_j \min(b_j, u)$ for catalog vertices and $p(\dagger) = \bar{b}$. Every selected-boundary arc satisfies*

$$T_{uv} + Q_{uv} = p(v) - p(u), \quad Q_{u\dagger} = p(\dagger) - p(u). \quad (13)$$

Thus its capacity is independent of the realization ceilings as long as $\tau_j \geq b_j$. For a feasible anchor a , $p(a) = a$.

Proof. Fix one history and an ordinary edge $u < v$. If $\tau_j < u$, then $b_j < u$, so both its contribution to the arc and $\min(b_j, v) - \min(b_j, u)$ are zero. If $u \leq \tau_j < v$, then $b_j < v$ and its exit-service capacity is $(b_j - u)_+ = \min(b_j, v) - \min(b_j, u)$. If $\tau_j \geq v$, its terminal increment is $[\min(b_j, v) - u]_+$, which equals the same difference. These cases partition the histories. Multiplication by the probabilities and summation prove the first identity. For the terminal edge, $\tau_j < u$ again implies $(b_j - u)_+ = 0$, giving $Q_{u\dagger} = \sum_j \pi_j (b_j - u)_+ = \bar{b} - p(u)$. Finally, feasibility implies $a \leq b_j$ for every history. $\square$

Changing a ceiling can still change an arc's payoff through its exit set. Proposition 3.6 asserts capacity invariance, not value continuity across a threshold. It also identifies the renewal reduction as a capacity-conservative configuration of the type used in the production model below.

3.2 A heterogeneous-reward primal extension

For heterogeneous rewards define the history-specific chord slope $\sigma_{uv}^j = [f_j(v) - f_j(u)]/(v - u)$. This gives the appropriate primal arc payoff. Simply replacing the common reward by different rewards in (10)

would be incorrect, because one chord slope no longer describes all histories. Instead, define the within-edge terminal allocation value

$$R_{uv}(x) = \max \left\{ \sum_{j: \tau_j \geq v} \pi_j \sigma_{uv}^j e_j : 0 \leq e_j \leq \delta_j(u, v), \sum_j \pi_j e_j = x \right\}, \quad 0 \leq x \leq T_{uv}. \quad (14)$$

The feasible within-edge polytope is nonempty and compact at each stated total, so its linear objective attains a maximum. Its value is continuous, increasing and concave, with marginal slopes drawn from the history-specific chord slopes. Set

$$\tilde{\Phi}_{uv}(x) = R_{uv}(\min(x, T_{uv})) - H_{uv}((x - T_{uv})_+). \quad (15)$$

The terminal edge still has payoff $-H_{u^\dagger}$. All terminal slopes are positive and all service marginal returns are nonpositive, so this joined payoff is concave.

Theorem 3.7 (History-level component packing). *For a common ordered catalog with ceiling-induced prefix eligibility and heterogeneous increasing concave terminal rewards, replace $f(a)$ in (12) by $\sum_j \pi_j f_j(a)$ and every Φ by $\tilde{\Phi}$. The resulting scalar-resource path identity is exact. Its capacities remain those in (11). For any feasible arc-resource vector, within-edge optimizers followed by history-level packing produce a feasible original policy with the same individual component totals and no smaller payoff.*

Proof. For a fixed book, introduce a component $e_{j,uv} \in [0, \delta_j(u, v)]$ for every eligible terminal segment, and service $z_j \in [0, (b_j - d_j)_+]$. Temporarily drop the requirement that a history's earlier components must be filled before its later ones. The relaxed objective, apart from charges, is

$$\sum_j \pi_j \left[f_j(a) + \sum_{(u,v)} \sigma_{uv}^j e_{j,uv} - h_j(z_j) \right], \quad \sum_j \pi_j \left[\sum_{(u,v)} e_{j,uv} + z_j \right] = B - a.$$

Every original canonical policy embeds in this relaxation by filling its eligible segments in order and then its service tail. Conversely, fix any relaxed component allocation and define $t_j = a + \sum_{(u,v)} e_{j,uv} + z_j$. The sum of that history's component capacities is exactly $b_j - a$, so $a \leq t_j \leq b_j$. Its components can be packed in their original order while preserving t_j : move service into any unfinished positive terminal segment, then move mass from a later positive segment into any unfinished earlier segment. Each move weakly improves payoff, because the history's terminal slopes are decreasing and positive and service cost is nondecreasing. This comparison is entirely within a history; it needs no common ordering across histories. At termination the components implement its canonical response at t_j . Proposition 2.1 supplies the adjacent lottery or sure-command service policy, with every realization constraint intact. The weighted promise is unchanged by packing. Thus relaxation and original problem have the same optimum.

Now regroup the relaxed components by selected edge, assigning z_j to the unique exit edge of history j . Groups share only the scalar total-resource equality; their individual component bounds are otherwise disjoint. For a fixed group resource x , optimal allocation fills positive terminal returns first, whose optimum is R_{uv} , and then minimizes the exit-set service cost H_{uv} . This is exactly (15). Maximizing over group

resources is therefore the same relaxation already proved exact. The component-capacity telescoping is unchanged. Within-edge optimizers followed by the preceding history-level packing provide the asserted constructive recovery. $\square$

In the quadratic specialization, R_{uv} is evaluated by sorting the eligible history slopes and filling weighted segment capacities. Its Lipschitz constant is at most $\max_j r_j$. The service part is bounded by $\max_j \gamma_j$. Hence the polynomial additive resource construction also extends to heterogeneous rewards with these new kernels. The heterogeneous merged-origin implementation in Section 6 evaluates these primal kernels directly. The predecessor readers retain the earlier common-reward comparison and its separate scope.

4 A Recognizable Class of Resource Paths

The renewal reduction is an instance of a more general allocation problem. Let $G = (V, E)$ be a finite directed acyclic graph with a common sink t , a set of possible starting vertices S , capacities $C_e \geq 0$, continuous edge payoffs ϕ_e on $[0, C_e]$, and source payoffs A_s . A decision selects a source-to-sink path P of at most $h \geq 1$ edges and allocations $0 \leq x_e \leq C_e$ satisfying $\sum_{e \in P} x_e = B_s$. Its value is $A_s + \sum_{e \in P} \phi_e(x_e)$, with any fixed edge or vertex charge included once in the corresponding path payoff. Retain only vertices reachable from a source and able to reach the sink. No ordered catalog, history, or eligibility prefix is required for this abstract problem.

Theorem 4.1 (Recognition and source-independent deficits). *The following are equivalent on the retained graph: (i) for every source, every source-to-sink path has the same total capacity; (ii) every vertex has a well-defined suffix capacity H_v , with $H_t = 0$ and $C_{uv} = H_u - H_v$ on every edge. The condition can be recognized in $O(|V| + |E|)$ arithmetic operations. When it fails, two paths from a common source with different total capacities form a directly checkable witness.*

Under these equivalent conditions, replacing x_e by $w_e = C_e - x_e$ yields a single terminal resource level independent of source and path if and only if $H_s - B_s = R$ is the same constant for every admissible source. In that case the exact optimum is

$$\max_{s, P} \left\{ A_s + \max_{0 \leq w_e \leq C_e, \sum_{e \in P} w_e = R} \sum_{e \in P} \phi_e(C_e - w_e) \right\}.$$

All sources can be initialized at deficit zero. Infeasible sources with $B_s \notin [0, H_s]$, or with no path within the edge limit, are excluded explicitly; if none remain, the problem is infeasible.

Proof. Condition (ii) telescopes, proving (i). Conversely fix a vertex v and a source-to- v prefix. Concatenating that prefix with any two v -to-sink suffixes gives valid paths, because the graph is acyclic. Condition (i) makes their suffix capacities equal. Define their common sum as H_v . Appending any edge uv to a suffix from v yields $H_u = C_{uv} + H_v$.

In reverse topological order set $H_t = 0$. At u , compute $C_{uv} + H_v$ for each successor and require all results to agree. This examines each edge once. Store a representative suffix at every vertex by successor pointers. If two values disagree, their edges followed by the stored suffixes have different capacities; prepend any

retained source prefix. The resulting witness has two explicit paths, whose capacity sums can be checked independently. Reachability and the minimum number of edges to the sink are also computed by topological passes.

On any path starting at s , $\sum C_e = H_s$, so $\sum x_e = B_s$ is equivalent to $\sum w_e = H_s - B_s$. This proves necessity and sufficiency for this particular complement coordinate, and is a bijection between the constrained allocation boxes. Continuations at a vertex depend only on that vertex, elapsed edge count, and accumulated deficit. Keeping the best prefix at each such state therefore loses no feasible complete path or value. The predecessor chain retains the actual source. The statement is not a claim that no other algorithm can merge states when capacity conservation fails. $\square$

Theorem 4.2 (Additive approximation on conservative paths). *Suppose the recognized instance has $R \geq 0$, every admissible source has $H_s \geq R$, and the centered deficit payoffs $g_e(w) = \phi_e(C_e - w) + \lambda_0 w$ are concave and K -Lipschitz for one common $K > 0$. With $\eta = \varepsilon/(2Kh)$, a layered path dynamic program on deficit multiples of η , accepting totals in $[R - h\eta, R]$, returns a feasible policy and a value interval of width at most ε . Apart from kernel evaluation, its arithmetic count is $O(h|E|Q \log(Q+1) + h|V|Q)$, where $Q = \lfloor R/\eta \rfloor + 1$. The result preserves the source, path, edge limit, and the exact original equality. Without concavity, the same guarantee holds for Lipschitz kernels using the direct $O(h|E|Q^2)$ convolution.*

Proof. Round the deficits of an optimal path down. At most h coordinates change, their total change is less than $h\eta$, and their centered payoff loss is at most $Kh\eta$. The best accepted grid value Z_η consequently gives upper bound $Z_\eta - \lambda_0 R + Kh\eta$. For a maximizing grid path, increase deficits to total R . Its total available capacity is $H_s \geq R$, so a greedy fill within residual edge capacities always completes this repair. Its total change is at most $h\eta$, giving a feasible lower value at least $Z_\eta - \lambda_0 R - Kh\eta$. Source payoffs and path charges do not change. Their difference is at most $2Kh\eta = \varepsilon$.

Initialize every feasible source at layer zero and deficit zero, and process edges between consecutive layers. Arbitrary holes in the finite support of a preceding row are permitted by Lemma 6.3, yielding the stated concave convolution count. Direct enumeration of both deficits proves the nonconcave count. At $R = 0$ there is one resource state and the method is exact. Numerical grid size, not its logarithm, appears in both bounds; rational bit complexity additionally depends on the encoding and exact evaluation of the supplied kernels. $\square$

For renewal design, $H_u = \bar{b} - p(u)$ and $B_u = B - u$ at a feasible anchor, for which $p(u) = u$. The common deficit is therefore $\bar{b} - B$, and $h = m$ including the terminal edge. Prefix eligibility and concave rewards are needed to derive and implement the renewal arc functions; they are not needed for the recognition identity. A separate class of examples is modular resource allocation along a precedence-compatible configuration path, with $C_{uv} = p(v) - p(u)$ and separable concave returns on installed modules. These examples instantiate the abstract theorem without interpreting vertices as renewal commands. They are structural examples, not field-calibrated applications. Multiple independent resource equalities require a multidimensional state and componentwise repair; the scalar complexity bound must not be carried over unchanged.

5 The Limited-Menu Complexity Frontier

Let $q_b = |\{b_j : j = 1, \dots, k\}|$ denote the number of distinct expected caps; the subscript distinguishes this count from a quadratic curvature. For a fixed command budget, enumerating all feasible books is polynomial with an exponent depending on that budget. Under a common reward, Theorem EC.2.3 reduces the useful book size to $\min\{m, 2q_b\}$, and its coefficient two is tight. The following result identifies why neither parameter generally removes the exponent dependence.

Theorem 5.1 (Parameterized hardness in menu size and cap count). *The rational finite-catalog decision problem is $W[1]$ -hard parameterized by $m + q_b$. Hardness holds with the fixed common reward $f(x) = x$, zero service costs, uniform strictly positive probabilities, $\tau_j = b_j$, nonnegative rational opening charges, and $m = q_b$. Consequently, unless $FPT = W[1]$, no exact algorithm has running time $F(m, q_b)\mathcal{L}^c$ for an arbitrary computable F and a constant c independent of the parameters, where $\mathcal{L}$ is binary input length. The same conclusion holds for parameterization by either m or q_b alone.*

The proof has two explicit stages. A group-sum encoding carries the consistency requirements of a multicolored clique. Dedicated baseline histories then make those group choices mandatory in the original renewal problem. Unlike a free augmentation under a nonbinding budget, every baseline must actually be installed in a threshold-achieving book.

Lemma 5.2 (Group-sum encoding). *From an instance of multicolored clique with r color classes, one can construct $g = r + \binom{r}{2}$ nonempty groups of positive integers $A_1, \dots, A_g$ and a positive integer W , of total binary length polynomial in the graph description, such that a selection of at most one integer from each group sums to W if and only if the graph has a multicolored clique. Every such sum uses exactly one integer from every group.*

Proof. Empty color classes or a color pair with no edge give an immediate no-instance. Otherwise number vertices by distinct codes in $\{1, \dots, n\}$. Create one group for each color and one for each unordered color pair. Use one digit for every group and two incidence digits for every color pair, in base $H > (2g + 1)(n + 1)$. The target has digit one at every group position and digit n at every incidence position.

A vertex of color i supplies digit one at its color-group position and its code at every incidence position involving that color. An edge joining codes u and v supplies digit one at its pair-group position and digits $n - u$ and $n - v$ at the two corresponding incidence positions. All unspecified digits are zero. Each encoded integer is positive because of its group digit. A selection of at most one integer per group has no carries: its digit sums are at most $gn < H$. Equality at the group digits forces every group to be represented. Equality at each incidence digit forces the chosen edge endpoint to equal the chosen vertex. Thus the selection is precisely a clique and its edges. Conversely such a clique supplies the required sum. There are $O(r^2)$ digits, each using $O(\log(rn))$ bits, and at most one encoded integer per vertex or edge. The construction is a parameterized reduction. Multicolored clique is $W[1]$ -hard: from ordinary r -clique make r color copies of every vertex and connect distinct copies exactly when their original vertices are adjacent; clique hardness follows by complementing the independent-set completeness result of Downey and Fellows (1995). $\square$

Proof of Theorem 5.1. Apply Lemma 5.2. A target outside $[0, \sum_i \max A_i]$ is an immediate no-instance and can be mapped to the fixed feasible renewal instance with one zero command, cap and promise zero, and threshold one. Otherwise set

$$M = \max_i \max A_i, \quad T = 2(g+1)M, \quad K = (2g+1)T, \quad \ell_i = \frac{i}{g+1}, \quad i = 1, \dots, g.$$

There are $2g+1$ equally likely histories: one with cap and ceiling zero, and, for each group i , two with caps and ceilings ℓ_i and $\ell_i + \max A_i/T$. The catalog contains zero, every baseline ℓ_i , and an optional command $\ell_i + w/T$ for each distinct $w \in A_i$. All points lie in $[0, 1]$. Options within group i lie strictly between its baseline and the next group's baseline; the last option is strictly below one. Equal integers within a group can be merged without changing the sum problem. Charge zero for zero and the baselines, and charge $w/(2K)$ for the option representing w . Set

$$m = 2g+1, \quad D_0 = \frac{2}{2g+1} \sum_i \ell_i, \quad B = D_0 + \frac{W}{K}, \quad \zeta = D_0 + \frac{W}{2K}. \quad (16)$$

The $2g+1$ caps are distinct, so $q_b = m$. The promise is feasible because $\bar{b} = D_0 + \sum_i \max A_i/K \geq B$ and zero is in the catalog.

Any feasible book must contain zero, since the zero-cap history has positive probability. Write $S(c)$ for the sum of the integers represented by its installed optional commands. With linear reward and free service, its exact gross value is

$$\min\{B, D(c)\}, \quad D(c) = \sum_j \pi_j \max(c \cap [0, b_j]).$$

If $B \leq D(c)$, lotteries between zero and each last eligible command attain the required aggregate terminal mean. If $B > D(c)$, use those last commands surely and fill the remaining promise with deterministic service; the available service capacity is $\bar{b} - D(c)$. These constructions obey the original realized ceilings, not only the expected caps.

For an upper bound only, augment the book by all baselines, ignoring the augmented count. Its terminal capacity is

$$D(c) \leq D_0 + \frac{1}{K} \sum_i \max(\{w : \ell_i + w/T \in c\} \cup \{0\}) \leq D_0 + \frac{S(c)}{K}.$$

The actual book, not the augmented comparison, pays charge $S(c)/(2K)$. Hence

$$V_c(B) \leq \min\{B, D_0 + S(c)/K\} - S(c)/(2K) = \zeta - \frac{|S(c) - W|}{2K}. \quad (17)$$

Suppose its value reaches ζ . Then $S(c) = W$ and both capacity inequalities must be equalities. All option weights are positive, so the second equality allows at most one optional command per group. The first equality forces every baseline into the actual book: if ℓ_i is absent, the dedicated history with cap ℓ_i has a strictly smaller last command, and adding ℓ_i strictly increases its positive-weight contribution to $D(c)$. Zero and the g baselines therefore consume $g+1$ slots. Lemma 5.2 now forces one option from each group. These

$2g + 1$ installed commands are within the original budget and encode a clique.

Conversely, choose the g options representing a clique together with zero and all baselines. This book uses exactly m commands, has $D(c) = D_0 + W/K = B$, and has net value ζ . This proves equivalence. The rational construction has polynomial bit length, and $m + q_b = 4g + 2 = O(r^2)$, proving the parameterized hardness statement. $\square$

This theorem does not assert $W[1]$ membership, a sharp exponent lower bound, strong NP-hardness, or hardness under uniform positive charges. It establishes a parameterized barrier for the original restricted model, rather than for a larger abstract path problem. Its fine rational scales also explain why it is consistent with the additive algorithm and the pseudo-polynomial lattice algorithm. Exact enumeration is XP, whereas an exponent independent of both menu size and cap count would collapse the stated parameterized classes. The full earlier SUBSET SUM construction and the tight cap-count proof are retained in the electronic companion.

6 Merged-Origin Resource-Deficit Paths

The capacity potential does more than guarantee repair after rounding. It identifies a resource coordinate in which every feasible anchor has the same initial state and every feasible book has the same terminal state. This removes the extra anchor loop in the resource algorithm. The construction applies to the heterogeneous kernels of Theorem 3.7.

6.1 An exact source-independent coordinate

Write

$$p(u) = \sum_j \pi_j \min\{b_j, u\}, \quad p(\dagger) = \bar{b}, \quad C_{uv} = T_{uv} + Q_{uv} = p(v) - p(u).$$

For the terminal edge, $C_{u\dagger} = \bar{b} - p(u)$. Every feasible anchor $a \leq \min\{B, \min_j b_j\}$ satisfies $p(a) = a$. If edge e is assigned resource x_e , define its unused capacity $w_e = C_e - x_e$ and its deficit payoff $\Psi_e(w) = \Phi_e(C_e - w)$. Opening charges are kept separate from Φ_e throughout. Let $R = \bar{b} - B$.

Theorem 6.1 (Merged-origin identity). *Under the common ordered catalog, prefix eligibility, and increasing concave history-specific rewards of Theorem 3.7, the original value equals*

$$\max_{\substack{c: 1 \leq |c| \leq m \\ a = \min c \leq \min(B, \min_j b_j)}} \left\{ \sum_j \pi_j f_j(a) - \sum_{z \in c} \rho(z) + \max_{\substack{0 \leq w_e \leq C_e \\ \sum_{e \in P(c)} w_e = R}} \sum_{e \in P(c)} \Psi_e(w_e) \right\}. \quad (18)$$

All anchors start at cumulative deficit zero and end at cumulative deficit R . Every feasible deficit vector on a chosen book yields a policy at the original promise, within the original command budget, with at least its displayed value. No anchor identifier is needed in a Bellman state once the current command and number of selected commands are known.

Proof. For a fixed book with anchor a , capacity conservation gives $\sum_e C_e = \bar{b} - a$. Hence

$$\sum_e x_e = B - a \iff \sum_e (C_e - x_e) = \bar{b} - B = R.$$

The transformation is a bijection between the two resource boxes intersected with their equalities. Apply the exact component-packing identity to obtain (18) and its recovery claim. Along a prefix ending at u , the cumulative deficit is

$$d = p(u) - a - \sum_{e \text{ in the prefix}} x_e.$$

At the anchor it is zero; traversing $u \rightarrow v$ adds $C_{uv} - x_{uv}$. Continuation payoffs and transitions depend only on u , the remaining command allowance, and d , not on the earlier anchor. A Bellman merge retains the best accumulated value among such prefixes. The recovered full path retains its own anchor through predecessor pointers. Partial paths need not themselves encode a canonical history policy: the full-path packing theorem supplies implementation after the equality is restored. $\square$

The same argument gives an abstract reuse principle. On an acyclic graph, suppose every resource capacity has the form $C_{uv} = p(v) - p(u) \geq 0$, all paths share a sink, and a source a requires resource $B - p(a)$ with $p(a) \leq B$. Complementing edge resources makes the total unused capacity $p(\dagger) - B$ independent of the source. Any additive path objective and edge-count bound then admit the same source merge. Concavity is needed for the approximation and fast convolution below, not for this algebraic identity. The renewal-specific content is the exact derivation of those capacities and payoffs from individual expected and realized constraints; an arbitrary constrained path need not have this potential structure.

6.2 Centered approximation and exact repair

For rational quadratic primitives put

$$r_{\max} = \max_j r_j, \quad G = \max_j \gamma_j, \quad \lambda_0 = (r_{\max} - G)/2, \quad K = (r_{\max} + G)/2 > 0.$$

Every supergradient of Φ_e lies between $-G$ and $r_{\max}$ on its interval. Consequently $g_e(w) = \Psi_e(w) + \lambda_0 w$ is concave and K -Lipschitz. Adding this common linear term changes every exactly feasible path value by the same constant $\lambda_0 R$; it does not change the optimizer.

For $\varepsilon > 0$ and $R > 0$, set

$$\eta = \frac{\varepsilon}{2Km}, \quad Q = \left\lceil \frac{R}{\eta} \right\rceil + 1, \quad \mathcal{I} = \{s \in \{0, \dots, Q-1\} : R - m\eta \leq s\eta \leq R\}. \quad (19)$$

Let $D_{\ell,v}(s)$ denote the best accumulated centered value at command a_v , with exactly ℓ selected commands

and cumulative deficit $s\eta$. Unreachable entries are $-\infty$. All feasible anchors are initialized simultaneously:

$$D_{1,i}(0) = \sum_j \pi_j f_j(a_i) - \rho_i, \quad a_i \leq \min\{B, \min_j b_j\}, \quad (20)$$

$$D_{\ell,v}(s) = \max_{\substack{u < v, 0 \leq t \leq s \\ t\eta \leq C_{uv}}} \{D_{\ell-1,u}(s-t) + g_{uv}(t\eta) - \rho_v\}, \quad (21)$$

$$S_\eta = \max_{\substack{1 \leq \ell \leq m, u, s \in \mathcal{I} \\ 0 \leq t \leq s, t\eta \leq C_{u^\dagger}}} \{D_{\ell,u}(s-t) + g_{u^\dagger}(t\eta)\}. \quad (22)$$

The tail does not install a new command. A book with s selected commands has s resource edges including that tail.

Theorem 6.2 (Centered deficit approximation). *Equations (20)–(22) and exact fixed-book recovery produce an original-instance interval*

$$\underline{V} \leq V^*(B) \leq S_\eta - \lambda_0 R + Km\eta, \quad S_\eta - \lambda_0 R + Km\eta - \underline{V} \leq \varepsilon. \quad (23)$$

The guarantee allows heterogeneous rewards, arbitrary full-catalog eligibility classes, arbitrary nonnegative charges, and the original budget. For $R = 0$ the same recurrence uses only deficit zero and is exact, including with nonzero service costs.

Proof. Round each deficit on an optimal continuous path down to a multiple of η . Its cumulative rounded deficit is no greater than R and differs from R by less than $m\eta$. Every prefix stays in the grid. Lipschitz continuity loses at most $Km\eta$ from the centered optimal value $V^*(B) + \lambda_0 R$. Thus $S_\eta \geq V^*(B) + \lambda_0 R - Km\eta$, proving the upper bound.

Conversely, recover a maximizing grid path. Increase its deficits, without exceeding their individual capacities, until their sum is R . This is always possible because the path's total capacity is $\bar{b} - a \geq \bar{b} - B = R$. The total increase is at most $m\eta$. The centered payoff decreases by at most $Km\eta$; after subtracting $\lambda_0 R$, the uncentered path is worth at least $S_\eta - \lambda_0 R - Km\eta$. Component packing gives an original feasible policy with no smaller value. Exact fixed-book allocation may improve it further, so the resulting feasible lower value satisfies the same bound. This proves (23). When $R = 0$, nonnegative deficits must all be zero, so there is no rounding, repair, or approximation error. $\square$

6.3 Convolution with holes and bit complexity

The merge can create unreachable gaps in a Bellman row. Concavity of the row itself is neither assumed nor generally true.

Lemma 6.3 (Concave convolution with arbitrary finite support). *Let $A(z) \in \mathbb{R} \cup \{-\infty\}$ be arbitrary on $0, \dots, Q-1$, and let $g(t)$ be a finite concave sequence on $0, \dots, L$. The smallest maximizing input index in*

$$M(s) = \max_{\max(0, s-L) \leq z \leq s} \{A(z) + g(s-z)\}$$

is nondecreasing across rows having at least one finite feasible column. All values and predecessors can be computed in $O(Q \log(Q + 1))$ comparisons and additions, including arbitrary holes in the support of A .

Proof. Delete columns with $A(z) = -\infty$ and rows whose feasible interval contains no remaining column. Concavity gives the anti-Monge inequality

$$[A(z) + g(s - z)] + [A(z') + g(s' - z')] \geq [A(z') + g(s - z')] + [A(z) + g(s' - z)]$$

whenever $s < s'$, $z < z'$, and the four entries are feasible. If two maximizing indices crossed in the wrong order, their crossed rectangle would be feasible because $0 \leq s - z \leq L$ is a staircase band. The inequality and the smallest-index tie rule would contradict that crossing. The maximizing indices are therefore monotone. Sorted finite columns permit feasible-row detection by interval searches. Divide-and-conquer row maximization scans a total $O(Q)$ candidate positions at each of $O(\log(Q + 1))$ levels, with shared endpoint candidates included. Empty rows are discarded before recursion; they are not allowed to impose an artificial predecessor bound on another row. $\square$

Proposition 6.4 (Arithmetic, storage, and verification). *For rational quadratic inputs the centered deficit algorithm has arithmetic bound*

$$O(kN^2Q \log(k + 1) + mN^2Q \log(Q + 1)). \quad (24)$$

Its cached implementation stores $O((N^2 + mN)Q + kN^2)$ rational entries and has bit complexity polynomial in binary input length and numerical Q . The extra factor N for separately enumerated anchors in (11) is absent. An independent checker can reconstruct all kernels and verify Bellman upper-potential inequalities in $O(k^2N^2Q + mN^2Q^2)$ rational operations, without using the optimizer's monotone search.

Proof. There are $O(N^2)$ ordinary and terminal edges. Sorted weighted terminal segments and sorted service saturation thresholds give piecewise-linear terminal profiles and piecewise-quadratic service profiles. Their endpoints and cumulative values are rational. Direct evaluation by capped allocation already gives the first term in (24); caching endpoints and binary searching them can reduce its constant and history dependence. For each selected count and edge, Lemma 6.3 gives the second term. The predecessor table has mNQ entries. Caching every edge grid adds N^2Q entries; retaining all component descriptions adds at most kN^2 . Thus the storage claim includes the edge cache rather than counting Bellman states alone.

A kernel value involves finite sums and products of input rationals, a grid rational, and at most one capped-service water-level division. Sums of at most k rational reciprocals have encoding length polynomial in the total input encoding. A path combines at most m such values. Therefore all represented integers have bit length polynomial in the input length and $\log(Q + 1)$; multiplying arithmetic counts by polynomial integer-arithmetic costs gives the stated bit bound. A common denominator may have exponentially large numerical value, which is not a polynomial-time claim in its binary encoding.

For independent verification, supplied Bellman values must dominate every feasible anchor and every predecessor-plus-edge transition. No transition into a claimed unreachable entry may be omitted. Every

accepted terminal combination must be covered by the reported terminal bound. These inequalities inductively bound every grid path; an attaining path certifies the grid optimum used in the repair calculation. A separately checked original lottery supplies the lower value. Direct service-threshold testing costs at most quadratic time in histories per kernel value; explicitly examining every predecessor deficit gives the stated conservative checker bound. The checker uses neither the forward optimizer nor its divide-and-conquer routine. $\square$

With $\delta = \varepsilon/K$ and $R \leq 1$, substitution gives a budget-quadratic, catalog-quadratic term of order $m^2 N^2 \delta^{-1} \log(m/\delta + 2)$ rather than its catalog-cubic predecessor. The guarantee is additive in signed net value. It is not polynomial in $\log(1/\varepsilon)$, and a successful optimization need not imply a cheap independent verification; both costs are measured separately.

7 Selecting Reward Prototypes

The oscillation guarantee becomes an optimization tool only after groups and prototypes are chosen. For catalog reward vectors define

$$d(f, h) = \max_{a \in \mathcal{A}} [f(a) - h(a)] - \min_{a \in \mathcal{A}} [f(a) - h(a)].$$

This is a pseudometric: symmetry follows by negating the difference, and the triangle inequality follows from subadditivity of maximum minus minimum. It is a metric after identifying vectors differing by a constant. Only histories with the same expected cap may share a prototype; ceilings, costs, probabilities, and promise remain unchanged.

For prototypes restricted to observed rewards, the minimum loss certificate with at most G groups is the cap-restricted weighted medoid problem

$$\min \sum_{j,r: b_j=b_r} \pi_j d(f_j, f_r) x_{jr}, \quad \sum_{r: b_r=b_j} x_{jr} = 1, \quad 0 \leq x_{jr} \leq z_r, \quad \sum_r z_r \leq G, \quad z_r \in \{0, 1\}.$$

For fixed selected centers a nearest-center assignment is optimal, so fractional assignment variables do not change the optimum. This is a standard facility-location model, not a new general clustering method. Its feasible upper bound supplies a valid loss certificate immediately, even if clustering is interrupted; a lower bound quantifies how much better this declared prototype family could be. Combined with Theorem EC.2.6, a clustering solution of cost Δ and a prototype-policy solution of error ξ give original regret at most $\Delta + \xi$ and an existence bound of $\min\{m, 2G\}$ commands.

Theorem 7.1 (Exact cap-preserving grouping frontier). *Suppose that within cap class b , rewards are $f_j(x) = \alpha_j + r_j x - \kappa_b x^2/2$, with $r_j \geq \kappa_b \geq 0$ and strict increase on the catalog. Among prototypes of this same class-specific curvature, the minimum oscillation loss for every group allowance through G can be computed by weighted-median dynamic programming in $O(k \log(k+1) + k^2 G + q_b G^2)$ rational operations and $O(k^2 + kG + q_b G)$ stored entries. This optimizes prototype selection, not the subsequent command-book search. Different expected caps are never merged.*

Proof. Additive constants and common curvature cancel in each difference. Thus $d(f_j, f_r) = (a_N - a_1)|r_j - r_r|$. For a fixed cap class, sort the slopes. For any chosen ordered center slopes, nearest-center regions on a line can be chosen as contiguous intervals; weights multiply their distances but do not affect the nearest-center choice for an individual point. Conversely, each interval is optimally represented by a weighted median, which can be chosen among its observed slopes. The allowed median preserves increasing concavity. Therefore an optimum has contiguous groups with observed medians.

For sorted indices $1, \dots, n_b$, let $C_b(i, j) = (a_N - a_1) \min_r \sum_{v=i}^j \pi_v |r_v - r|$. Prefix sums of weights and weighted slopes, with a median pointer that moves only forward as j increases for fixed i , compute all interval costs in $O(n_b^2)$ operations. Define

$$F_b(t, j) = \min_{t-1 \leq i < j} \{F_b(t-1, i) + C_b(i+1, j)\}, \quad F_b(0, 0) = 0,$$

with other infeasible states infinite. This yields every cap-class frontier in $O(Gn_b^2)$ operations. Convolve these frontiers over cap classes, allocating at least one group to each nonempty class and at most G in total. This takes $O(q_b G^2)$ operations. Backtracking yields centers and assignments. The storage bound includes interval costs and backpointers. Summing over classes gives the result. With $G < q_b$ the declared cap-preserving grouping problem is infeasible; with $G \geq k$ its loss is zero. $\square$

For arbitrary reward curves the medoid formulation remains valid, but the one-dimensional dynamic program is not asserted. No statistical estimation guarantee is implicit in this calculation. The tests compare the computed frontier with exhaustive medoid choices, and the resulting loss remains in the same reward-minus-cost units as the original objective.

8 Computational Evidence and Decision Cost

The study separates three questions: whether the reductions and recovery identities agree with independent exact calculations; how a matched end-to-end allowance is consumed by solving and checking; and which numerical parameters obstruct the theoretical algorithms. It does not test adoption by a provider. The operational architecture and every earlier experiment remain in the retained complete R58 readers.

8.1 Exact structural checks

The parameterized reduction is tested on 4,240 candidate books, including books that omit baseline commands or install multiple options from a group. Fifty-one values are cross-checked against the existing rational fixed-book allocator. Forty-eight small graph instances compare clique existence with the full no-carry group encoding. The cap-preserving prototype algorithm agrees with exhaustive medoid selection on 160 group-budget frontiers. These finite tests supplement the general proofs; they are neither a proof by experiment nor independent evidence of real-world cap or reward clustering.

8.2 Matched resources and stronger formulation

The new frozen panel contains eight seeds for each of eight one-factor cells. The baseline has 24 histories, 17 catalog points and a budget of four. Other cells vary the catalog to 33 and 65, histories to 96, budget to two and eight, and additive tolerance from one hundredth to one twentieth and one five-hundredth. Probabilities are nonuniform normalized positive integers; promises cycle through one quarter, one half and three quarters of aggregate capacity. All inputs, distinct cap counts, joint reward-type counts, probability-denominator bit lengths and predicted grid sizes are recorded per instance. Eight additional group-sum instances use an exact target and two to five groups. A structured exhaustive oracle supplies their exact optimum, without treating equal numbers of lexicographic books as equal work.

Each of the 72 instances is assigned price search, the deficit algorithm, enumeration, and the direct mixed-integer convex formulation in Section EC.6. All 288 requests receive three seconds from process creation through independent checking, with an internal optimization cutoff at 1.8 seconds. Execution is sequential and method order is deterministically shuffled within each instance. The deficit implementation retains the declared 600,000-state guard; rational lattice sizes are reported both numerically and in bits. A state guard or deadline is an outcome, not a removed observation. This short-budget panel measures an explicit resource regime, not asymptotic runtime or the best performance achievable with longer solver tuning.

The SCIP formulation retains the exact quadratic service epigraph rather than a piecewise-linear service envelope. Its formulation error is zero; its numerical feasibility tolerance and global bounds are not rational proofs. A selected book is reconstructed by exact allocation, and that original-space policy is checked independently. The common accuracy target is an additive difference in the same net-value units. Rational and numerical target attainment occupy separate columns; the latter uses a stated reporting tolerance of 10^{-8} . An optimizer returning a narrow interval after the total deadline does not count as a completed decision. Independent checking, serialization, startup and model construction all consume the common allowance.

The new panel contains 288 method requests. Price search branches in 7 of 72 cases, with maximum recorded depth 16. Table 2 reports completed outputs and target attainment rather than optimizer status alone. Table 3 retains end-to-end deadlines, grid rejections, and checking costs. The heterogeneous grid and binary-encoded challenge cases intentionally expose numerical-state and proof-cost limits. A common time allowance does not turn a numerical global bound into a rational certificate. These results identify method-dependent tradeoffs on this generator; they do not establish universal solver superiority or field performance.

8.3 Screening and limits of interpretation

The earlier matched screening study remains a negative economic result: preprocessing improved total optimization, serialization and checking time in none of 18 pairs, with a median ratio of about 1.604. A safe removal rule is therefore not automatically an economical preprocessing rule. We retain its mathematical guarantee and all raw observations, but do not cite removal counts as evidence of end-to-end speedup. Screening is not activated in the new primary panel. A deployment decision would need to compare its fixed

overhead with downstream savings on the actual workload.

The new panel varies more dimensions and adds an exact convex formulation, but remains a deterministic synthetic engineering study. It is not independently generated field evidence, a statistically representative sample of applications, or proof that branching depth is controlled in general. The parameterized theorem is the explanation for why a bounded command budget does not itself yield a fixed-parameter algorithm; experiments cannot replace that distinction. Complete row-level outputs, checker high-water memory and resource-limit records support inspection beyond aggregate medians.

9 Conclusion

Endogenous command support can be separated from continuous allocation without discarding individual feasibility. The selected-boundary construction gives implementable path payoffs; capacity conservation identifies when sources can share a deficit coordinate; and repair on the same path converts a numerical grid into an exact-promise policy. The original model is nevertheless parameterized-hard jointly in command budget and cap count. Small-type compression therefore gives a meaningful support theorem and an XP algorithm, not a general fixed-parameter solution.

This frontier leaves useful design choices rather than a single universal solver. A common-curvature prototype frontier trades a certified payoff loss against a smaller common catalog. Conservative paths admit additive guarantees even when exact optimization is obstructed by binary encoding. A direct convex formulation and rational path certificates supply different forms of evidence and incur different costs. The preserved unfavorable screening result illustrates why safe preprocessing and economical decision support must be evaluated separately. The mathematical scope remains the original accepted-promise problem; extending feasibility inputs to uncertain or endogenous distributions requires an additional model, not an unproved transfer of the present guarantees.

Code and data. The accompanying code-and-data package contains the complete current source, frozen inputs, all requested outcomes, independent checkers, build validation and a content-preservation map. The previous complete article and companion remain available as retained readers, so reorganizing the main argument removes no prior proof or empirical record. Timing reproduction must use a separate directory; checking the committed evidence is a read-only operation and does not replace the original study.

References

- Aggarwal A, Klawe MM, Moran S, Shor P, Wilber R (1987) Geometric applications of a matrix-searching algorithm. *Algorithmica* 2:195–208. doi:10.1007/BF01840359.
- Arya V, Garg N, Khandekar R, Meyerson A, Munagala K, Pandit V (2004) Local search heuristics for k -median and facility location problems. *SIAM Journal on Computing* 33(3):544–562. doi:10.1137/S0097539702416402.
- Babai M, Gonczarowski YA, Nisan N (2016) The menu-size complexity of revenue approximation. Author manuscript, arXiv:1604.06580.
- Beasley JE, Christofides N (1989) An algorithm for the resource constrained shortest path problem. *Networks* 19(4):379–394. doi:10.1002/net.3230190402.
- Bolusani S, et al. (2024) The SCIP Optimization Suite 9.0. ZIB Report 24-02, Zuse Institute Berlin. The execution manifest records the version actually run; this citation documents the solver framework.
- Downey RG, Fellows MR (1995) Fixed-parameter tractability and completeness II: On completeness for $W[1]$. *Theoretical Computer Science* 141(1–2):109–131. doi:10.1016/0304-3975(94)00097-3.
- Fisher ML (1981) The Lagrangian relaxation method for solving integer programming problems. *Management Science* 27(1):1–18. doi:10.1287/mnsc.27.1.1.
- Handler GY, Zang I (1980) A dual algorithm for the constrained shortest path problem. *Networks* 10(4):293–309. doi:10.1002/net.3230100403.
- Hart S, Nisan N (2013) Selling multiple correlated goods: Revenue maximization and menu-size complexity. Author manuscript, arXiv:1304.6116.
- Hassin R (1992) Approximation schemes for the restricted shortest path problem. *Mathematics of Operations Research* 17(1):36–42. doi:10.1287/moor.17.1.36.
- Karp RM (1972) Reducibility among combinatorial problems. Miller RE, Thatcher JW, eds. *Complexity of Computer Computations* (Plenum Press, New York), 85–103. doi:10.1007/978-1-4684-2001-2_9.
- Lorenz DH, Raz D (2001) A simple efficient approximation scheme for the restricted shortest path problem. *Operations Research Letters* 28(5):213–219. doi:10.1016/S0167-6377(01)00069-4.
- Maher S, Miltenberger M, Pedroso JP, Rehfeldt D, Schwarz R, Serrano F (2016) PySCIPOpt: Mathematical programming in Python with the SCIP optimization suite. *Mathematical Software—ICMS 2016*, 301–307. doi:10.1007/978-3-319-42432-3_37.
- Mohajerin Esfahani P, Kuhn D (2018) Data-driven distributionally robust optimization using the Wasserstein metric: Performance guarantees and tractable reformulations. *Mathematical Programming* 171:115–166. doi:10.1007/s10107-017-1172-1.

- Pagès G, Wilbertz B (2012) Intrinsic stationarity for vector quantization: Foundation of dual quantization. Author manuscript, arXiv:1010.4642v2, March 27.
- Patriksson M (2008) A survey on the continuous nonlinear resource allocation problem. *European Journal of Operational Research* 185(1):1–46. doi:10.1016/j.ejor.2006.12.006.
- Rusmevichientong P, Shen ZJM, Shmoys DB (2010) Dynamic assortment optimization with a multinomial logit choice model and capacity constraint. *Operations Research* 58(6):1666–1680. doi:10.1287/opre.1100.0866.
- Virtanen P, Gommers R, Oliphant TE, et al. (2020) SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods* 17:261–272. doi:10.1038/s41592-019-0686-2.
- Winkler G (1988) Extreme points of moment sets. *Mathematics of Operations Research* 13(4):581–587. doi:10.1287/moor.13.4.581.
- Wu X (1991) Optimal quantization by matrix searching. *Journal of Algorithms* 12(4):663–673. doi:10.1016/0196-6774(91)90039-2.
- Zhang H (2012) Solving an infinite horizon adverse selection model through finite policy graphs. *Operations Research* 60(4):850–864. doi:10.1287/opre.1120.1056.

Table 1: Theorem-level positioning.

Antecedent	Established conclusion	Distinction in this model
Constrained paths	Resource-state search, dual bounds, approximation	Exact elimination of individual lottery and realization constraints; source merge and original-equality repair.
Continuous allocation	Optimal allocation on a fixed separable system	The command support and its fixed costs are endogenous.
Moment support	Small distributional support under moment conditions	A paid union of conditional supports, with history-specific caps and ceilings.
Auction menus	Complexity of incentive-compatible outcome-payment alternatives	Controlled terminal commands, observed histories, no incentive constraints.
Parameterized complexity	Hardness via parameter-preserving reductions	Budget-consuming baseline histories yield hardness in menu size and cap count within the original model.
Clustering and robustness	Prototype or ambiguity-set guarantees	Cap-preserving oscillation cost maps directly into original-feasible menu loss.

Table 2: Matched three-second end-to-end panel (72 instances per method)

Method	Checked	Rational target	Numerical target	Time (s)	RSS (MiB)
Price	72	71	0	0.339	26.7
Deficit	55	5	0	1.852	32.4
Enumeration	72	12	0	1.827	26.7
Scip	72	0	72	0.527	68.6

Checked counts completed independent checks within the total allowance. SCIP checks only its rationally reconstructed feasible policy; its global bound remains numerical. Numerical targets use the same additive target with a stated 10^{-8} reporting tolerance, and are not exact proofs. Time includes process startup, construction, optimization, serialization and checking. RSS is a process high-water mark. No unfinished output counts as a certified target.

Table 3: Independent-checking cost and retained resource limits

Method	Check time (s)	Check RSS (MiB)	Deadline	State limit	Error
Price	0.0521	26.7	0	0	0
Deficit	0.0022	34.8	17	0	0
Enumeration	0.0025	26.7	0	0	0
Scip	0.0062	68.6	0	0	0

Checking medians are conditional on a recorded completed checking phase, not on all requests. The checker runs in the same process; its RSS includes inherited optimizer allocations and is a conservative end-to-end memory measure, not incremental checker memory. Deadline includes killed jobs; internal optimization interruption may instead return a checked fallback interval.